```
  Enum.map(messages, fn message ->
    %Schema.Conversation{
      contact: message.contact,
      messages: [message]
    }
  end)
end

def load_conversation_with(contact) do
  messages = Query.SmsMessageStore.load_messages_with(contact)
  %Schema.Conversation{contact: contact, messages: messages}
end
```

These functions use the queries that we wrote earlier, but they each wrap the result in a helpful `Conversation` struct. This is entirely optional, but it leads to a nicer function for the rest of the application.

And that's it for our application's query layer! We aren't actually doing anything with it yet, but the next chapter pulls everything together to make our application take shape.

Wrapping Up

Ecto is a library for data mapping and database queries. It takes a different design approach than Ruby's ActiveRecord library, but somehow it ends up feeling familiar. Many of the concepts and terms are similar between the libraries, so you'll be right at home as you learn to use it. Ecto is used to query the database, translate data into your application, and change data from your application.

Ecto schemas are modules that define your application's data structure. Ecto uses a schema to know how to map data from the database into an Elixir struct. You can define associations in your Ecto schema, such as one-to-one, one-to-many, and many-to-many associations. These associations are standardized across database libraries, so they feel identical to ActiveRecord associations.

Migrations are used to create your database tables, fields, and indices. Ecto has a robust data migration format that's similar to ActiveRecord migrations. But you have to be careful with migrations. Anytime that you change the structure of a database, you run the risk of causing problems in your application or with your database performance. These risks exist in any migration library, so they aren't an Ecto issue. There are best practices that you can follow to reduce the risks.